

Lecture 08

SCF / Affine Loop

M/N/K tile loop and DMA ordering for AI/NPU compiler

M/N/K Tiled MatMul Loop Nest

for m0 in 0..M step TM

output row tile

for n0 in 0..N step TN

output col tile

zero C tile accumulator

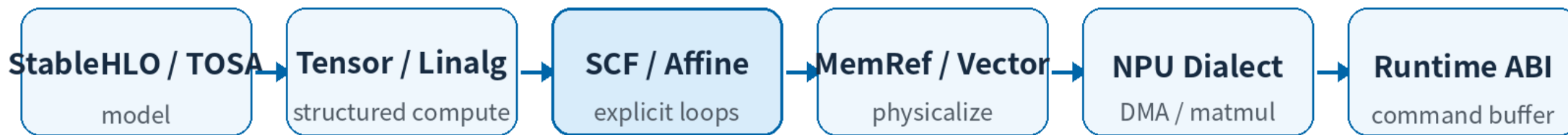
accumulator SRAM

for k0 in 0..K step TK

reduction tile

오늘의 위치

Lecture 08: SCF / Affine Loop 위치



NPU compiler 질문

- M/N/K loop order는 어떤 reuse를 만드는가?
- tile shape가 SRAM/accumulator/DMA burst에 맞는가?
- DMA와 compute를 overlap할 수 있는가?
- edge tile과 dynamic shape policy는 무엇인가?

Learning Objectives

SCF 와 Affine dialect 의 역할과 차이를 설명한다 .

linalg.matmul 을 M/N/K tiled loop nest 로 전개한다 .

tile size 가 SRAM footprint, accumulator, DMA burst 에 미치는 영향을 계산한다 .

DMA load - wait/barrier - compute - store 순서를 NPU schedule 로 작성한다 .

SCF/Affine loop 기반 regression test 항목을 설계한다 .

왜 Loop 가 NPU 성능을 좌우하는가 ?

NPU peak TOPS 는 MAC array utilization 이 높을 때만 의미가 있다 .

Loop order 는 A/B/C reuse, SRAM residency, DRAM burst, edge tile overhead 를 결정한다 .

Tile size 는 SRAM capacity, bank conflict, accumulator width, vector lane 과 연결된다 .

DMA 와 compute 를 overlap 하지 못하면 memory-bound 로 떨어진다 .

따라서 loop IR 은 compiler decision 을 노출하고 검증하는 핵심 단계다 .

Abstraction Boundary

Linalg: structured compute 와 indexing semantics 보존

SCF/Affine: explicit iteration space 와 loop-carried state 노출

MemRef: physical buffer, memory space, layout 로 이동

Vector/NPU Dialect: micro-kernel, DMA, barrier, command packet 으로 이동

Runtime ABI: command buffer 와 tensor descriptor 로 실행

SCF Dialect 핵심

SCF = Structured Control Flow

주요 op: `scf.for`, `scf.if`, `scf.parallel`, `scf.forall`, `scf.yield`

Region/Block 기반으로 loop body 와 branch body 를 구조적으로 표현한다 .

dynamic shape, runtime condition, scheduling skeleton 표현에 적합하다 .

NPU command sequence 를 만들기 전의 readable loop IR 로 유용하다 .

scf.for Anatomy

```
%c0 = arith.constant 0 : index
%c128 = arith.constant 128 : index
%c1 = arith.constant 1 : index

scf.for %i = %c0 to %c128 step %c1 {
  %a = memref.load %A[%i] : memref<128xf32>
  memref.store %a, %B[%i] : memref<128xf32>
}
```

scf.for Field Checklist

lower bound: loop 시작 index

upper bound: exclusive end index

step: iteration stride

induction variable: body 에서 사용하는 SSA value

body region: 반복 실행되는 operation list

terminator: loop-carried value 가 있으면 scf.yield 가 필수

Loop-carried Value

```
%sum = scf.for %i = %c0 to %c128 step %c1
  iter_args(%acc = %zero) -> (f32) {
    %a = memref.load %A[%i] : memref<128xf32>
    %next = arith.addf %acc, %a : f32
    scf.yield %next : f32
  }

// NPU 관점 : accumulator state, token,
// software pipeline state를 loop-carried SSA로 표현 가능
```

scf.if / scf.parallel / scf.forall

scf.if: runtime shape, edge tile, fallback path decision 에 사용 가능

scf.parallel: 독립 loop dimension 의 병렬성 표현

scf.forall: thread/block mapping 성격의 structured parallel loop 표현

NPU 에서는 hardware queue, core cluster, tile-level parallelism 과 연결 가능
단 , 정확한 mapping 은 target dialect/Transform dialect 와 함께 결정한다 .

Affine Dialect 핵심

Affine loop 와 affine map 으로 analyzable index computation 을 표현한다 .

affine.for, affine.apply, affine.load/store, affine.min/max 가 핵심이다 .

static bounds 와 affine memory access 가 있으면 dependence analysis 와 loop transform 에 유리하다 .

polyhedral-style optimization 을 적용하기 좋은 representation 이다 .

복잡한 data-dependent branch 는 affine 표현에 맞지 않을 수 있다 .

Affine Map: Dimension vs Symbol

```
#map = affine_map<(d0, d1)[s0] -> (d0 * 64 + d1 + s0)>  
  
// d0, d1: loop induction variable 처럼 변하는 dimension  
// s0: function argument, shape, stride 처럼 map 이 설정될 때 bound 되는 symbol  
  
%idx = affine.apply #map(%m0, %mi)[%base]
```

SCF vs Affine Decision

SCF vs Affine Decision Matrix

SCF Dialect

- structured control flow
- dynamic bounds / runtime branch
- scf.for, scf.if, scf.forall
- NPU command scheduling skeleton
- less restrictive, less analyzable

Affine Dialect

- affine loop/access/maps
- static loop analysis
- affine.for, affine.apply
- tiling/interchange/dependence
- more restrictive, more analyzable

Rule: dynamic scheduling은 SCF, 정적 loop/access 분석은 Affine으로 두는 것이 일반적이다.

MatMul Equation

$C[m,n] = \sum_k A[m,k] * B[k,n]$

M loop: output row tile 을 선택

N loop: output column tile 을 선택

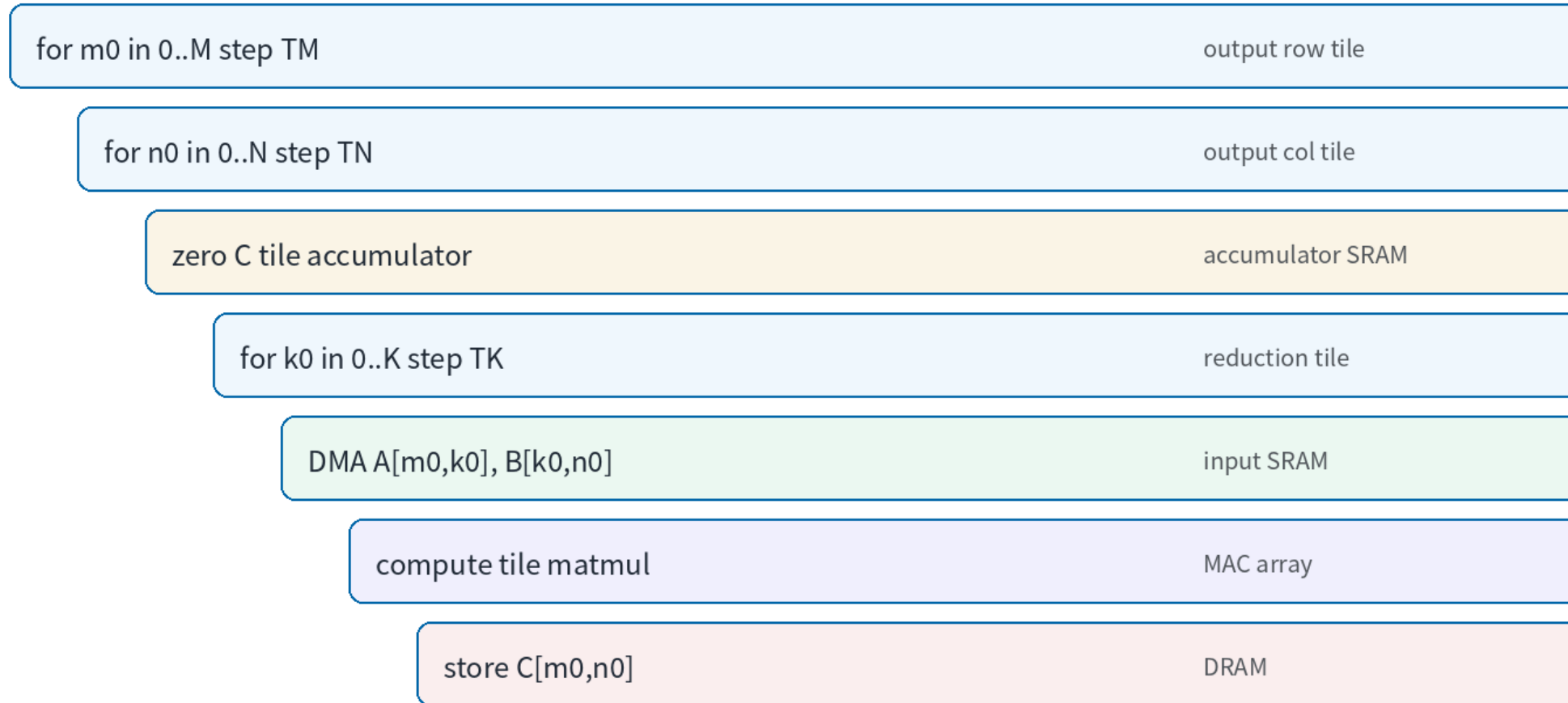
K loop: reduction tile 을 stream

Accumulator lifetime: 하나의 C tile 에 대해 모든 K tile 이 끝날 때까지 유지

DMA lifetime: A/B input tile 은 각 K tile 마다 교체 가능

M/N/K Tiled Loop Nest

M/N/K Tiled MatMul Loop Nest



SCF Tiled MatMul Pseudo IR

```
scf.for %m0 = %c0 to %M step %TM {  
  scf.for %n0 = %c0 to %N step %TN {  
    "npu.acc.zero"(%m0, %n0, %TM, %TN) : (...) -> ()  
    scf.for %k0 = %c0 to %K step %TK {  
      "npu.dma.load_a"(%A, %m0, %k0, %TM, %TK) : (...) -> ()  
      "npu.dma.load_b"(%B, %k0, %n0, %TK, %TN) : (...) -> ()  
      "npu.barrier.wait_dma"() : () -> ()  
      "npu.matmul.tile"(%m0, %n0, %k0, %TM, %TN, %TK) : (...) -> ()  
    }  
    "npu.dma.store_c"(%C, %m0, %n0, %TM, %TN) : (...) -> ()  
  }  
}
```

Affine Tiled MatMul Skeleton

```
affine.for %m0 = 0 to 128 step 32 {  
  affine.for %n0 = 0 to 128 step 64 {  
    affine.for %k0 = 0 to 256 step 64 {  
      affine.for %mi = 0 to 32 {  
        affine.for %ni = 0 to 64 {  
          affine.for %ki = 0 to 64 {  
            %a = affine.load %A[%m0 + %mi, %k0 + %ki] : memref<128x256xf32>  
            %b = affine.load %B[%k0 + %ki, %n0 + %ni] : memref<256x128xf32>  
            // C += A * B  
          }  
        }  
      }  
    }  
  }  
}
```

Loop Order 와 Data Reuse

m0/n0 outer loop: 하나의 C tile 을 accumulator 에 유지한다 .

k0 inner loop: A/B tile 을 stream 하면서 C tile 에 누적한다 .

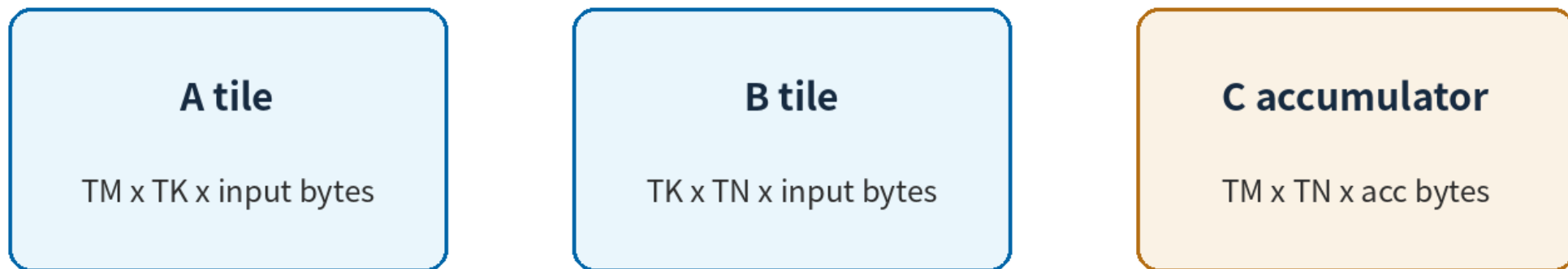
M-major ordering: A row tile reuse 와 C row locality 에 유리할 수 있다 .

N-major ordering: B column tile reuse 또는 output store pattern 에 유리할 수 있다 .

Target NPU 의 SRAM bank 와 DMA burst pattern 을 보고 결정해야 한다 .

Tile Footprint 계산

SRAM Tile Footprint



Example: TM=32, TN=64, TK=64, int8 input, int32 accumulator

- $A = 32 * 64 * 1 = 2 \text{ KB}$
- $B = 64 * 64 * 1 = 4 \text{ KB}$
- $\text{Acc} = 32 * 64 * 4 = 8 \text{ KB}$
- Double-buffered A/B + Acc = $2 * (2 + 4) + 8 = 20 \text{ KB}$
- Add alignment/tag/bank overhead before accepting tile.

Tile Size Selection Heuristics

TM/TN 은 MAC array output tile shape 와 accumulator bank 에 맞춘다 .

TK 는 A/B streaming 효율과 reduction loop overhead 를 균형화한다 .

input tile double buffering 을 고려해 SRAM budget 을 계산한다 .

DMA burst alignment, page boundary, bank conflict 를 별도 constraint 로 둔다 .

edge tile 이 너무 많으면 tile size 또는 padding policy 를 재검토한다 .

Edge Tile Policy

Problem: $M/N/K$ 가 tile size 의 배수가 아닐 수 있다 .

Option 1: effective tile size = $\min(\text{tile}, \text{remaining})$

Option 2: mask-based compute and store

Option 3: pad input/output to tile boundary

Option 4: fallback micro-kernel or CPU path

Compiler 는 read/write out-of-bounds 를 반드시 방지해야 한다 .

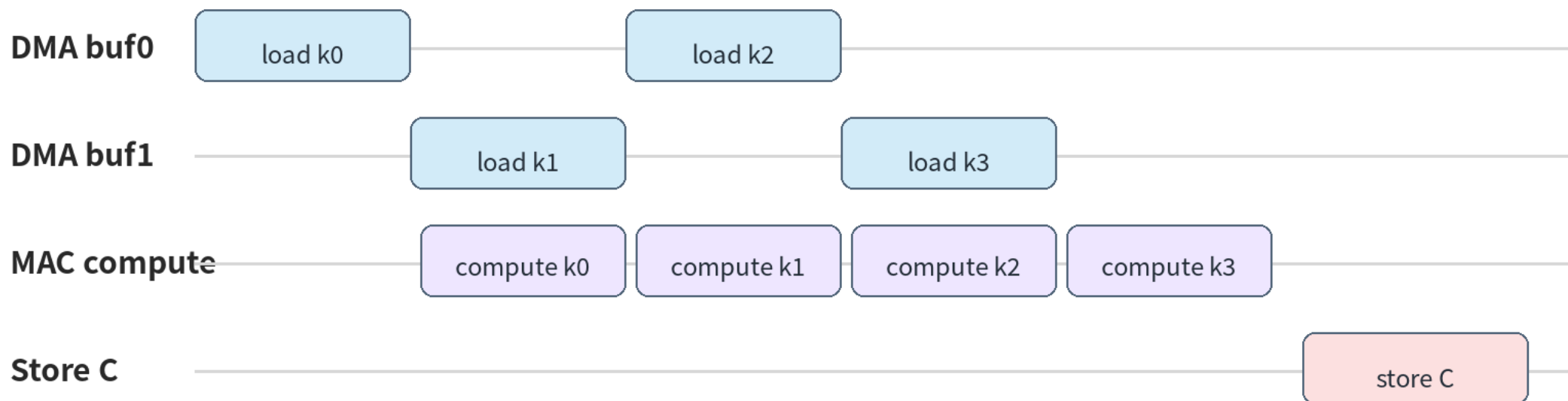
DMA Baseline Schedule

1. DMA A tile from DRAM to SRAM
2. DMA B tile from DRAM to SRAM
3. wait/barrier until DMA completes
4. compute $C \text{ tile} += A \text{ tile} * B \text{ tile}$
5. repeat for next K tile
6. store completed C tile to DRAM

Baseline 은 쉽지만 compute 와 DMA 가 직렬화되어 latency hiding 이 약하다 .

Double-buffered DMA Timeline

Double-buffered DMA / Compute Timeline



Scheduling rule: preload next K tile while MAC consumes current tile. Wait only before true consume.

- no compute-before-DMA-complete
- no DMA overwrite-before-consume
- store only after all K tiles accumulated
- edge tile must preserve valid read/write ranges

Barrier Placement Rules

DMA wait 는 compute 가 해당 tile 을 소비하기 직전에 둔다 .

compute 가 읽는 SRAM buffer 를 DMA 가 overwrite 하면 안 된다 .

store 는 accumulator 가 모든 K tile 을 누적한 뒤에만 시작한다 .

too early wait: overlap 손실

too late wait: correctness hazard

각 barrier 는 compiler test 에서 ordering contract 로 검증해야 한다 .

DMA Pseudo IR

```
// preload k0 into buffer 0
"npu.dma.start"(%A, %B, %c0) {buffer = 0 : i32} : (...) -> ()
"npu.dma.wait"() {buffer = 0 : i32} : () -> ()

scf.for %k0 = %c0 to %K step %TK {
  "npu.dma.start_next_if_any"(%A, %B, %k0) : (...) -> ()
  "npu.matmul.consume_current_buffer"(%k0) : (index) -> ()
  "npu.dma.wait_next_if_any"(%k0) : (index) -> ()
}

"npu.dma.store"(%C) : (...) -> ()
```

SCF/Affine -> NPU Dialect Lowering

SCF/Affine loop: tile-level iteration and scheduling intent

Bufferization/MemRef: physical buffer, memory space, layout

Vector/Vectorization: inner micro-kernel shape

NPU kernel dialect: matmul tile, dma, barrier, accumulator ops

NPU runtime dialect: command buffer emit and ABI lowering

Regression Test Checklist

Loop structure: m0/n0/k0 order and tile sizes

Accumulator lifetime: zero before K loop, store after K loop

DMA ordering: load A/B before matmul, wait before consume

Double-buffer policy: no overwrite-before-consume

Edge tile policy: min/mask/padding/fallback correctness

Types and memory spaces: input, accumulator, output consistency

FileCheck Template

```
// RUN: mlir-opt %s --canonicalize | FileCheck %s

// CHECK-LABEL: func.func @scf_tiled_matmul_pseudo
// CHECK: scf.for
// CHECK: scf.for
// CHECK: "npu.acc.zero"
// CHECK: scf.for
// CHECK: "npu.dma.load_a"
// CHECK: "npu.dma.load_b"
// CHECK: "npu.barrier.wait_dma"
// CHECK: "npu.matmul.tile"
// CHECK: "npu.dma.store_c"
```

Exercise A: scf.for Annotation

01_scf_for_basic.mlir 을 열고 각 loop field 를 표시한다 .

body region 의 operation list 를 적는다 .

loop-carried value 예제에서 init value 와 yielded value 를 연결한다 .

NPU compiler 에서 loop-carried state 로 표현할 수 있는 것을 3 개 제안한다 .

Exercise B: Tiled MatMul Loop

Shape: $A[128,256] \times B[256,128] \rightarrow C[128,128]$

Tile: $TM=32, TN=64, TK=64$

$m0/n0/k0$ trip count 를 계산한다 .

SCF pseudo loop 를 작성한다 .

Accumulator zero, update, store 위치를 표시한다 .

Exercise C: DMA Schedule

Baseline schedule 과 double-buffer schedule 을 각각 작성한다 .

각 step 의 DMA action, compute action, wait/barrier 를 표로 쓴다 .

wait 가 빨라지는 경우와 늦어지는 경우의 문제를 설명한다 .

SRAM buffer 0/1 의 ownership 상태를 각 step 에 표시한다 .

Common Mistakes

K loop 밖에서 accumulator 를 store 하지 않고 중간 store 를 해버림

DMA wait 를 load 직후에 넣어 compute/DMA overlap 을 없앴

edge tile 에서 out-of-bounds read/write 를 허용함

SCF 와 Affine 의 표현 제약을 고려하지 않고 non-affine index 를 affine 에 넣음

tile size 는 맞지만 SRAM bank/alignment constraint 를 빠뜨림

Homework

M=256, N=192, K=384 에 대해 tile 후보 3 개를 비교한다 .

(32,64,64), (64,64,64), (32,128,64)

각 후보의 trip count, SRAM footprint, expected reuse 를 계산한다 .

가장 좋은 후보를 선택하고 SCF pseudo IR + DMA schedule table 을 작성한다 .

FileCheck CHECK line 최소 5 개를 작성한다 .

Takeaways

SCF/Affine 은 NPU compiler 가 schedule 을 드러내는 첫 explicit loop 단계다 .

SCF 는 dynamic/runtime scheduling skeleton 에 좋고 , Affine 은 static loop/access analysis 에 좋다 .

M/N/K loop order 는 C accumulator reuse 와 A/B DMA streaming 을 결정한다 .

tile size 는 SRAM footprint 와 MAC utilization 의 contract 다 .

DMA/barrier ordering 은 성능과 correctness 를 동시에 결정한다 .

References

MLIR SCF Dialect: <https://mlir.llvm.org/docs/Dialects/SCFDialect/>

MLIR Affine Dialect: <https://mlir.llvm.org/docs/Dialects/Affine/>

MLIR MemRef Dialect: <https://mlir.llvm.org/docs/Dialects/MemRef/>

MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>